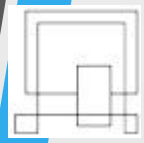
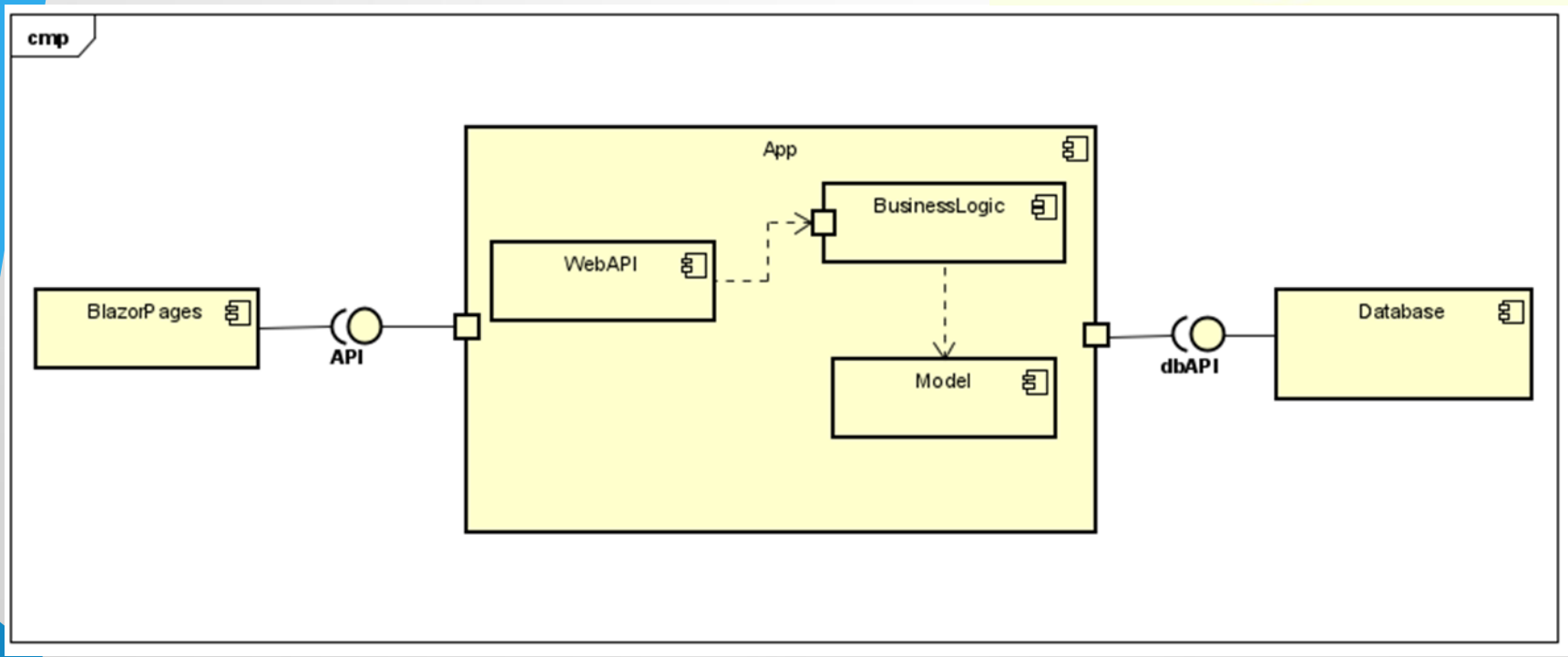
VIA University College



Programming 3

Autumn 2026





https://eng.zemosolabs.com/when-not-to-autowire-in-spring-spring-boot-93e6a01cb793

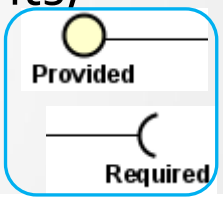
Learning Objectives

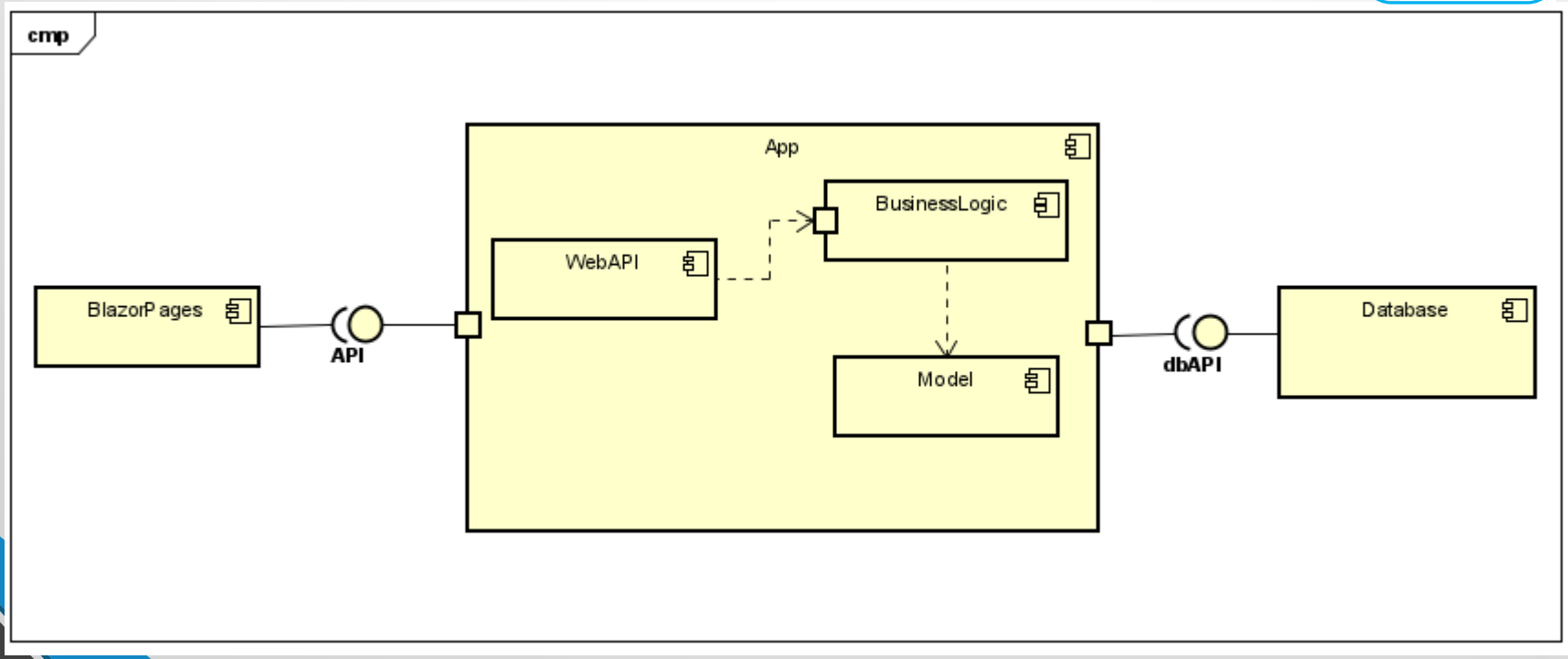
- By the end of today's session, you should be able to:
  - ✓ explain distributed components and the concept of containers
  - ✓ explain Dependency Injection, roles in dependency injection, and name some popular containers
  - ✓ explain dependency injection in Java using Google Guice.
  - ✓ explain Inversion of Control and implement dependency injection in Spring

Component Diagram

A unit of composition with contractually specified interfaces and explicit context dependencies only. [Szyperski 2002]

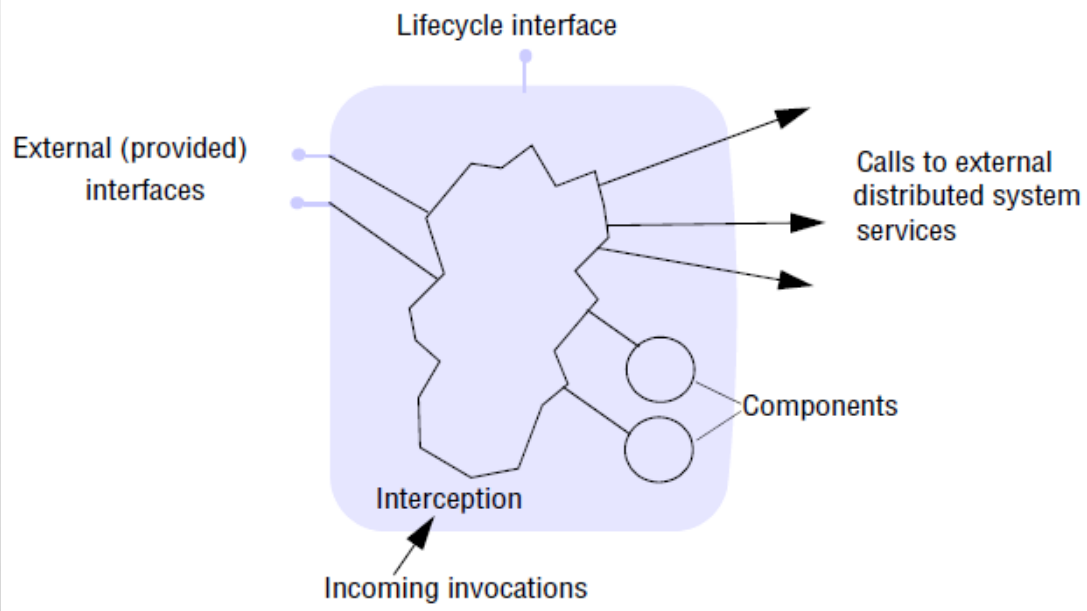
- An example software architecture consisting of components, interfaces and connections between interfaces.





Containers

- The concept of containers is central to component-based middleware.
- Containers support the three-tier model
  - Front-end client – a container for application logic – system services that manage persistent data
- Provide a managed server-side hosting environments for components



Dependency Injection

IoC - Inversion of Control: a design methodology used to build a loosely coupled system in software engineering by inverting the control of flow from your main program to some other entity or framework.

DI - Dependency injection is a programming technique that makes a class independent of its dependencies. **Injection** refers to the process of passing the dependent object into a software component.

- Goal: remove this dependency by separating the usage from the creation of the object.
- This reduces the amount of required boilerplate code and improves flexibility.
- DI is one of the ways to implement IoC.
- 4 roles in dependency injection
  - Interface - that's used by the client and implemented by the service
  - Service - the service you want to use.
  - Injector - which creates a service instance and injects it into the client
  - Client - that uses the service

Types of Dependency Injection

Three types of dependency injection

- **constructor injection**: the dependencies are provided through a class constructor.
  - one needs to pass the dependent object through a public constructor of a client object
  - Constructor injection can control the order of instantiation and consequently reduce the risk of circular dependency
- **setter injection**: the client exposes a setter method that the injector uses to inject the dependency.
  - Setter injection is a perfect fit for optional or conditional dependency
- **interface injection**: the dependency provides an injector method that will inject the dependency into any client passed to it.
  - Technically, interface injection and setter injection are the same. They both use some sort of method to inject dependency. However, for interface injection, the method is defined by objects which provide the dependency

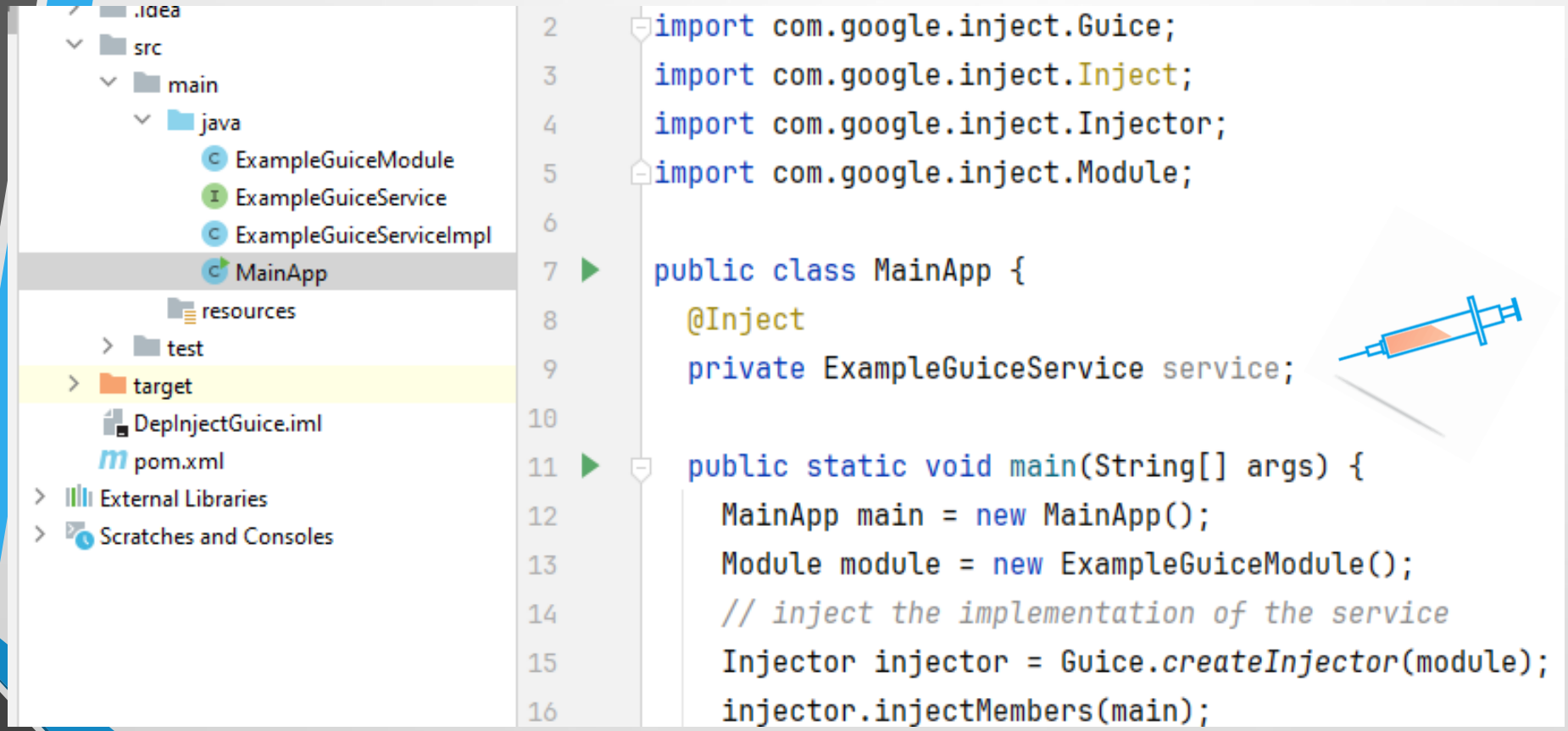
Dependency Injection Containers

A container takes care of creating, configuring, and managing objects.

Some popular examples of dependency injection frameworks

- Guice - lightweight dependency injection framework for Java by Google.
- Spring * - dependency injection in the Spring framework.
- Weld - Context Dependency Injection for Java specification
- Play framework - makes it easy to build web applications with Java and Scala
- Dagger - dependency injector for Android and Java

Dependency Injection in Guice



Types of dependency injection

- **Constructor Injection**

```
public class AppConsumer {
  private NotificationService notificationService;

  //Constructor level Injection
  @Inject
  public AppConsumer(NotificationService service) {
    this.notificationService = service;
  }
}
```

- **Method injection**

```
private NotificationService notificationService;
//Setter Injection
@Inject
public void setService(NotificationService service) {
  this.notificationService = service;
}
```

- **Field injection**

```
@Inject private NotificationService notificationService;
```

Client

```
import com.google.inject.Guice;
import com.google.inject.Injector;

public class NotificationClient
{
  public static void main(String[] args)
  {
    Injector injector = Guice.createInjector(new
AppointmentModule());
    AppointmentConsumer app =
injector.getInstance(AppointmentConsumer.class);
    app.sendNotofication("Your appointment is due in
1 day", "78798788");
  }
}
```

Dependency Injection in Spring

- How do I implement dependency injection in a Spring Boot application?
- We can use Spring out-of-the-box DI support via annotations.



Types of dependency injection

- **Constructor Injection**

```
@RestController
public class ProductPackagerController {
    private ProductPackager rabbitMQService;

    public ProductPackagerController(ProductPackager rabbitMQService) {
        this.rabbitMQService = rabbitMQService;
    }
}
```

- **Method/setter injection**

```
private ProductPackager rabbitMQService;
//Setter Injection
@Autowired
public void setService(ProductPackager rabbitMQService) {
  this. rabbitMQService = rabbitMQService;
}
```

- **Field injection**

```
@Autowired
private Room room;
```

Constructor vs Field vs Setter

Feature	Constructor Injection	Field Injection	Setter Injection
Reliability	More reliable because of immutable variables	Less reliable	Less reliable
Maintainability	Constructor injection helps in identifying the list of dependencies and can easily be used to make the code more readable	Since Autowired fields can easily mix up with regular fields, it gets difficult to track the dependencies of the class	Same as Field Injection
Testability	Mocking the dependencies become easier as they can be mocked and passed using constructor even without creating Spring Context which enables faster and cleaner tests	Mocking becomes challenging as creating mock beans and actual beans and shuffling between them becomes tedious	Mocking is easy as mock object can directly be set
Consistency	More consistent as whole application uses similar coding and injection pattern	Less consistent as Field Injections can be randomly included anywhere in the code	Less consistent as Setter Injections can be randomly included anywhere in the code

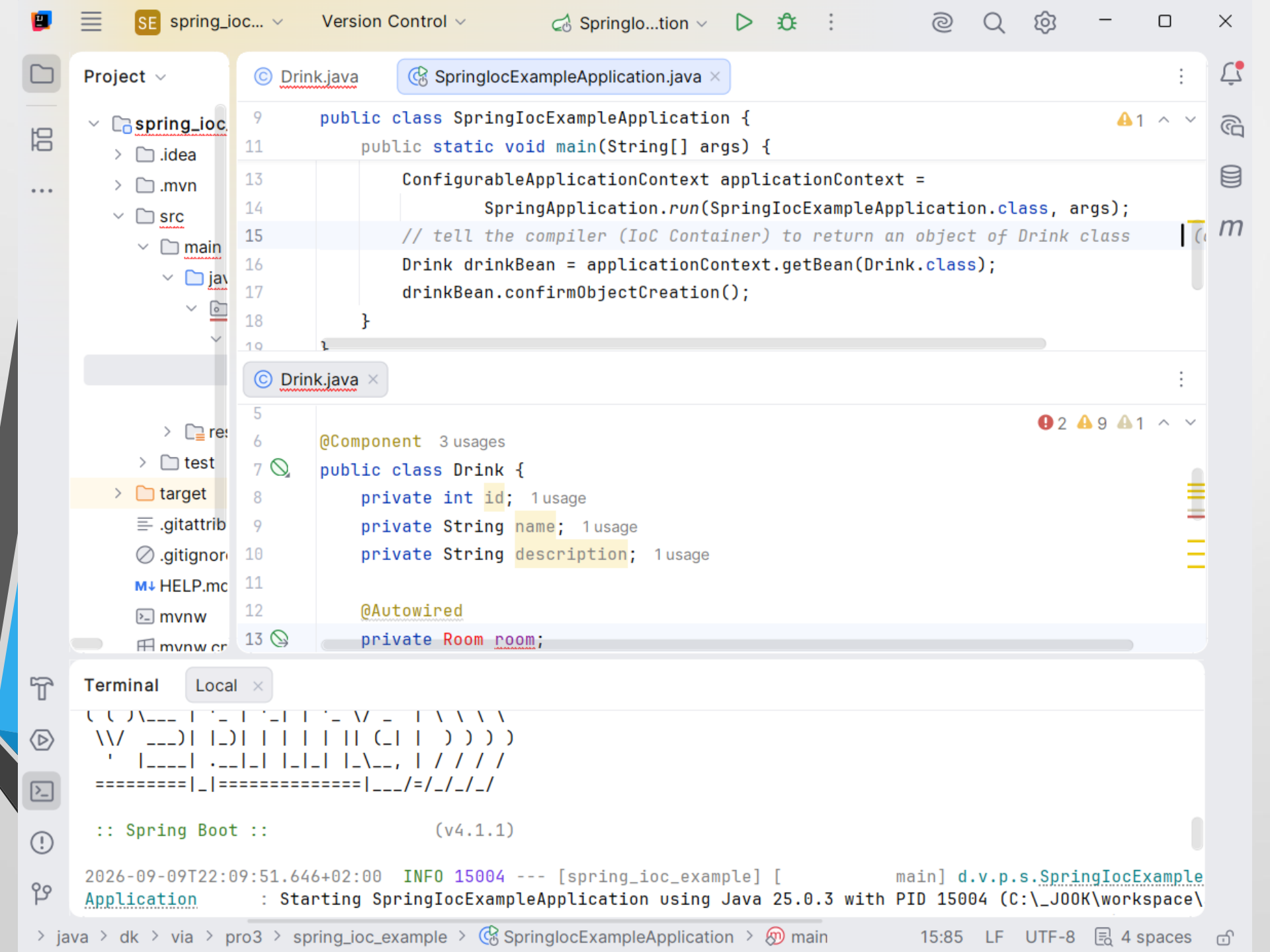
Constructor Injection should always be the first choice because it is reliable

https://airtable.com/app3Y9txv2k3EGICU/shrzXJoAJzXvERfEg/tblnumzbfmQX9QlGh

Dependency Injection in Spring

- How do I implement dependency injection in a Spring Boot application?

Let's write some code…


spring_ioc...
Version Control
Springlo...tion
Project
spring_ioc
.idea
.mvn
src
main
jav
res
test
target
.gitattrib
.gitignor
HELP.mc
mvnw
mvnw.cr
Drink.java
SpringIocExampleApplication.java
9 public class SpringIocExampleApplication {
11 public static void main(String[] args) {
13 ConfigurableApplicationContext applicationContext =
14 SpringApplication.run(SpringIocExampleApplication.class, args);
15 // tell the compiler (IoC Container) to return an object of Drink class
16 Drink drinkBean = applicationContext.getBean(Drink.class);
17 drinkBean.confirmObjectCreation();
18 }
Drink.java
6 @Component 3 usages
7 public class Drink {
8 private int id; 1 usage
9 private String name; 1 usage
10 private String description; 1 usage
12 @Autowired
13 private Room room;
Terminal
Local
:: Spring Boot :: (v4.1.1)
2026-09-09T22:09:51.646+02:00 INFO 15004 --- [spring_ioc_example] [main] d.v.p.s.SpringIocExample
Application : Starting SpringIocExampleApplication using Java 25.0.3 with PID 15004 (C:_JOOK\workspace\
java > dk > via > pro3 > spring_ioc_example > SpringIocExampleApplication > main
15:85 LF UTF-8 4 spaces